Federated Learning Orchestration Architecture

Core Synchronization Challenge

You need to coordinate **4 distinct phases** across distributed containers:

1. **Training Phase** - K selected nodes train locally
2. **Upload Phase** - Nodes upload to blockchain
3. **Verification Phase** - Blockchain consensus
4. **Aggregation Phase** - Hospital aggregates → Multi-institutional ensemble

---

Architecture Options

Option 1: Centralized Orchestrator (Recommended)

How it works:

- **Central Server (h_node)** acts as the "conductor"
- Maintains a **global state machine** for FL rounds
- Uses **Redis Pub/Sub** for command distribution
- Tracks participant readiness via heartbeats + Redis

State Machine:

```
IDLE → SELECT_CLIENTS → BROADCAST_WEIGHTS → TRAINING
→ WAIT_FOR_UPLOADS → AGGREGATION → UPDATE_GLOBAL
→ MULTI_INST_SYNC → IDLE (next round)
```

Synchronization Mechanism:

- Hospital publishes: `ROUND_START:{round_num}` to Redis channel
- Edge nodes subscribe, receive signal, begin training
- Nodes set Redis key: `node:{id}:status = "training"`
- Hospital polls Redis: "Are all K nodes done?"
- When all done → trigger aggregation
- Blockchain naturally serializes uploads (first-come-first-serve)

Pros:

- Simple mental model
- Easy debugging (single source of truth)
- Your Redis is already there

Cons:

- Single point of failure (mitigated with Redis persistence)

Option 2: Event-Driven with Message Queue

How it works:

- Use **Redis Streams** (or RabbitMQ/Kafka if you want overkill)
- Each component produces/consumes events
- No central coordinator - components react to events

Event Flow:

```
Hospital publishes: ROUND_INITIATED
→ Nodes consume, start training
→ Nodes produce: TRAINING_COMPLETE
→ Blockchain consumes: MODEL_UPLOADED
→ Hospital consumes: AGGREGATION_TRIGGER
→ Hospital produces: GLOBAL_MODEL_READY
```

Pros:

- Scalable, decoupled
- Natural fit for async workflows

Cons:

- More complex (event ordering, idempotency)
- Harder to debug "where are we in the round?"

Option 3: Hybrid - Coordinator + Barriers

How it works:

- Central server issues commands

- Use **barrier synchronization** with Redis
- Nodes "check in" at barriers before proceeding

Barrier Pattern:

```
# Pseudocode
round_barrier = f"round:{round_num}:training_complete"
redis.sadd(round_barrier, node_id)  # Node adds itself
count = redis.scard(round_barrier)   # Hospital checks count

if count >= K_CLIENTS:
    redis.publish("START_AGGREGATION", round_num)
```

Pros:

- Combines control + flexibility
- Clear synchronization points

Practical Implementation Strategy

1. Training Round Coordination

Hospital (Central Server):

```
LOOP for T rounds:
  1. Select K random nodes (from heartbeat registry)
  2. Publish to Redis: "train:round:{r}:nodes" = [node_0, node_3, node_5]
  3. Publish signal: "START_ROUND_{r}"
  4. Wait for barrier: K nodes report "TRAINING_DONE"
  5. Trigger next phase
```

Edge Nodes:

```
SUBSCRIBE to "START_ROUND_*"
ON_MESSAGE:
  - Check if my ID is in selected list
  - If yes: load global weights, train for E epochs
  - Set Redis: node:{id}:status = "uploading"
  - Upload model to blockchain
  - Set Redis: node:{id}:status = "upload_complete"
```

2. Upload & Verification Timing

Already mostly solved!

- Blockchain `/submit_local_model` is synchronous
- It blocks until consensus completes
- Natural serialization - no race conditions

Enhancement:

- Hospital watches blockchain events (or polls ledger)
- When `len(hospital_0_chain) == previous + K`, all models uploaded
- Trigger aggregation

---

3. Aggregation Phase

Single Hospital:

```
WAIT until blockchain has K new blocks
FETCH K model hashes from blockchain
DOWNLOAD models (from where? Need model storage!)
RUN FedAvg on K models
SAVE global model
PUBLISH to Redis: "global_model_ready:round_{r}"
UPDATE blockchain with aggregated model hash
```

Multi-Institutional:

```
WAIT until ALL 3 hospitals publish "global_model_ready"
FETCH 3 hospital global models
RUN ensemble aggregation (weighted average?)
APPEND to multi_inst_chain
BROADCAST new global weights to ALL nodes
```

---

4. Timing Mechanisms

Avoid hardcoded sleeps! Use:

A) Polling with Exponential Backoff

```
while not all_nodes_ready():
    time.sleep(backoff)
    backoff = min(backoff * 2, MAX_BACKOFF)
```

B) Redis Keyspace Notifications

- Enable in Redis: `CONFIG SET notify-keyspace-events Ex`
- Hospital subscribes to key expiration events
- Node sets: `SETEX node:5:ready 120 "done"` → auto-expires if node crashes

C) Timeout Policies

- If node doesn't complete in 10 minutes → drop it, continue with K-1
- Add `timeout` field to round state

---

5. State Persistence

Critical for crash recovery:

Hospital stores in Redis:

```
{
  "current_round": 3,
  "round_state": "TRAINING",
  "selected_nodes": [0, 2, 4],
  "completed_nodes": [0, 2],
  "started_at": 1738972800,
  "global_model_path": "/models/global_round_3.pth"
}
```

On crash/restart:

- Read state from Redis
- Resume where left off (or retry round)

Minimal User Interaction - Startup Sequence

One-command startup:

```
docker-compose up -d
# Wait 10s for all services to be healthy
docker exec h_node_0 python orchestrator.py --rounds 10 --epochs 20
```

Orchestrator does:

1. Wait for all nodes to send first heartbeat (health check)
2. Initialize global model, broadcast to nodes
3. Start Round 1 automatically
4. Loop through all rounds
5. Exit when done, save final model

Recommended Implementation Phases

Phase 1 (MVP)

- Centralized orchestrator in `h_node`
- Redis Pub/Sub for commands
- Redis hash for state tracking
- Polling with barriers for synchronization

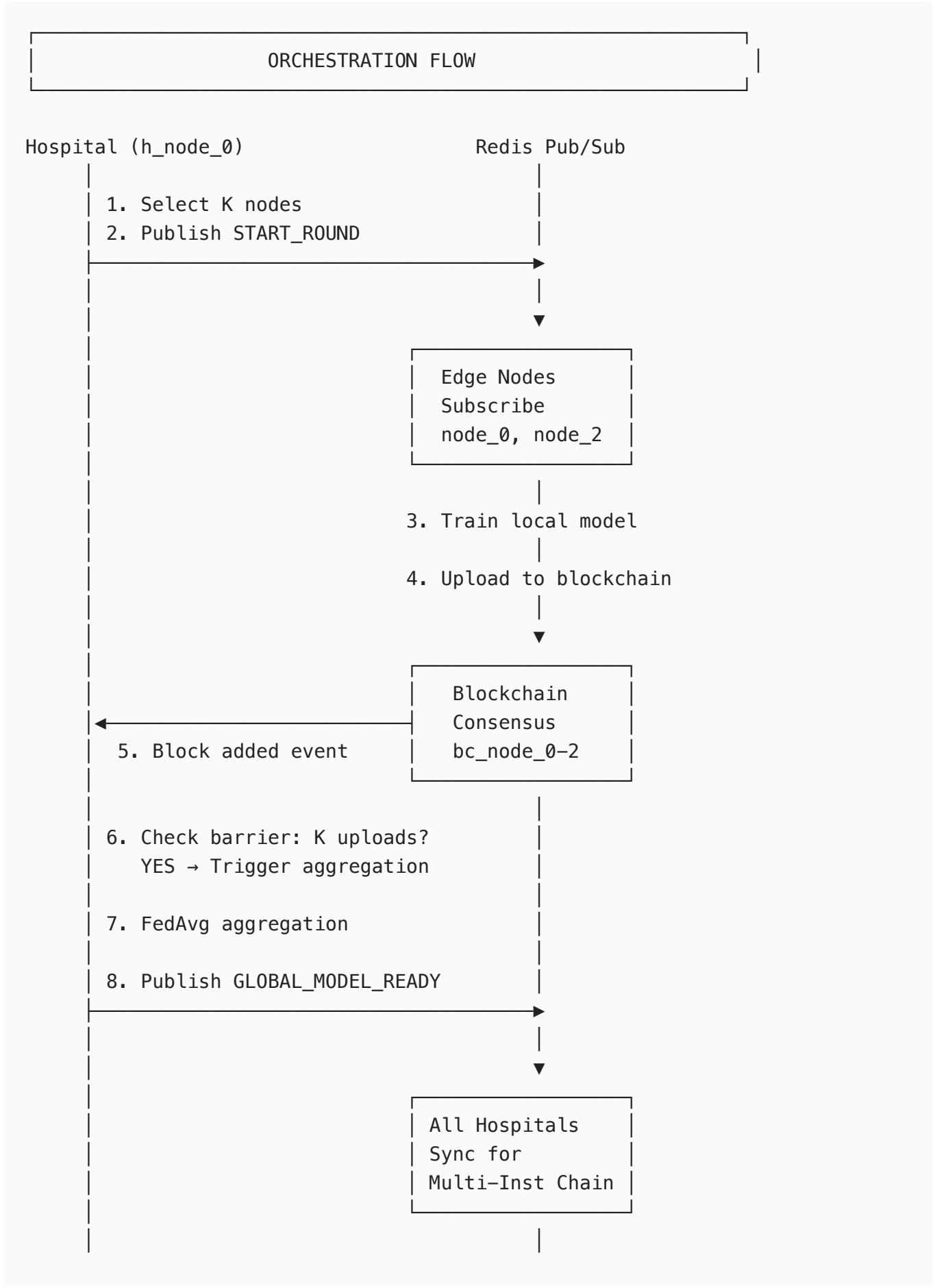
Phase 2 (Production)

- Add timeout handling
- Event-driven architecture with Redis Streams
- Health checks + automatic node replacement
- Dashboard to visualize round progress

Key Files to Create

- `h_node/orchestrator.py` - Main FL coordinator
- `node/training_agent.py` - Listens for commands, trains on signal
- `shared/state_manager.py` - Redis state utilities
- `bc_node/event_listener.py` - Notify hospital when blocks added

Architecture Diagram



```
        | 9. Next Round                          |
        └────────────────────────────────────────┘
```

Redis Key Schema

State Keys

- `fl:current_round` → Integer (current round number)
- `fl:round:{r}:state` → String ("TRAINING" | "UPLOADING" | "AGGREGATING")
- `fl:round:{r}:selected_nodes` → Set (node IDs)
- `fl:round:{r}:completed_nodes` → Set (node IDs that finished)
- `fl:round:{r}:started_at` → Timestamp

Node Status Keys

- `node:{id}:status` → Hash {state, round, timestamp, metrics}
- `node:{id}:last_heartbeat` → Timestamp (with TTL)

Barriers

- `barrier:round:{r}:training_complete` → Set (node IDs)
- `barrier:round:{r}:upload_complete` → Set (node IDs)

Global Model

- `global_model:round:{r}:path` → String (file path)
- `global_model:round:{r}:hash` → String (SHA256)

Pub/Sub Channels

- `fl_commands` → Commands (START_ROUND, STOP_TRAINING, etc.)
- `fl_events` → Events (ROUND_COMPLETE, MODEL_UPLOADED, etc.)
- `hospital:{id}:events` → Hospital-specific events

Critical Implementation Notes

1. **Model Storage Problem**: Currently models aren't stored, only hashes. Need to add:
   - S3/MinIO for model storage

- Or: store models locally, reference by hash in blockchain

2. **Network Partitions**: What if node loses connection mid-training?
   - Use heartbeat TTL to detect dead nodes
   - Remove from selected set, continue with K-1
3. **Byzantine Nodes**: What if node uploads corrupted model?
   - Blockchain consensus handles this (hash mismatch = reject)
   - Add reputation scores in multi_inst_chain
4. **Race Conditions**: Multiple hospitals might select same nodes
   - Add node reservation system in Redis
   - Hospital claims nodes before training starts
5. **Aggregation Strategy**: How to fetch models for aggregation?
   - Option A: Nodes also send to hospital directly (duplicate upload)
   - Option B: Hospital fetches from blockchain storage (needs implementation)
   - Option C: Nodes upload to shared storage, blockchain stores reference

---

Next Steps

1. Decide on model storage strategy (S3 vs local vs hybrid)
2. Implement `orchestrator.py` with basic state machine
3. Add Redis Pub/Sub listeners to edge nodes
4. Create barrier synchronization utilities
5. Test with 2 nodes, 1 hospital, 1 round
6. Scale up to full system